The Chooser and Saver APIs provide a number of advantages over a normal OAuth2 flow for this simple file sharing use case:

- The app author doesn’t have to decide ahead of time what resource it needs to access. Instead they just tell Dropbox that they need a file to open or to save data to and Dropbox lets the user decide which file to use. The app never gets to see a list of the user’s other files at all.
- Because the app is not requesting long-term access to the user’s account, there is no need for a consent page to ensure the user knows what access they are granted. Selecting a file in the UI implicitly indicates consent and because the scope is so fine-grained, the risks of abuse are much lower.
- The UI is implemented by Dropbox and so is consistent for every app and web page that uses the API. Little details like the “Recent” menu item work consistently across all apps.

For these use cases, capabilities provide a very intuitive and natural user experience that is also significantly more secure than the alternatives. It’s often assumed that there is a natural trade-off between security and usability: the more secure a system is, the harder it must be to use. Capabilities seem to defy this conventional wisdom, because moving to a more fine-grained management of permissions allows more convenient patterns of interaction. The user chooses the files they want to work with, and the system grants the